

Akshit Demo Web Shop Playwright Capstone - Final Project Report

Generated on: June 2, 2026

1. Project Overview

This capstone project is a Playwright JavaScript automation framework for the Demo Web Shop e-commerce application.

Website under test:

`https://demowebshop.tricentis.com`

The project validates important customer-facing e-commerce flows using automated UI tests. It follows a Page Object Model structure, uses Playwright fixtures for reusable page objects, supports cross-browser execution, and includes Allure reporting with GitHub Pages publishing support.

2. Project Objective

The objective of this project is to automate functional test coverage for a demo e-commerce website across multiple service areas and browsers.

Main goals:

- Build a maintainable Playwright automation framework.
- Implement 8 functional service modules.
- Automate 15 test cases per service.
- Execute tests across Chromium, Firefox, and WebKit.
- Generate Playwright HTML and Allure reports.
- Support GitHub Actions execution and GitHub Pages report hosting.

3. Application Under Test

Application name:

Demo Web Shop

Application type:

E-commerce demo website

Main flows covered:

- Authentication and account access
- Product catalog and search
- Cart controls

- Wishlist and comparison features
- User profile links
- Address and shipping information
- Checkout and payment journey labels
- Customer support and information pages

4. Technology Stack

Component	Tool
Automation framework	Playwright Test
Programming language	JavaScript
Runtime	Node.js
Design pattern	Page Object Model
Test runner	Playwright
Browser support	Chromium, Firefox, WebKit
Reporting	Playwright HTML Report, Allure Report
CI/CD	GitHub Actions
Report hosting	GitHub Pages

5. Project Structure

```
Akshit_Capstone
|-- .github
|   |-- workflows
|       |-- allure-report.yml
|-- api
|   |-- apiClient.js
|-- data
|   |-- demoWebShopData.js
|-- docs
|   |-- Akshit_Capstone_Final_Project_Report.md
|   |-- Capstone_Day_Wise_Schedule.docx
|   |-- Demo_Web_Shop_Capstone_Testing_Plan.docx
|   |-- Demo_Web_Shop_Capstone_Testing_Plan.pdf
|   |-- Playwright_Notes.pdf
|-- fixtures
|   |-- baseFixture.js
|-- pages
|   |-- addressShipping.page.js
|   |-- authentication.page.js
|   |-- base.page.js
|   |-- cart.page.js
|   |-- checkoutPayment.page.js
|   |-- customerSupport.page.js
|   |-- product.page.js
|   |-- userProfile.page.js
|   |-- wishlistCompare.page.js
|-- screenshots
|-- tests
|   |-- address-shipping
|   |-- authentication
|   |-- cart
|   |-- checkout-payment
|   |-- customer-support
|   |-- product
|   |-- user-profile
|   |-- wishlist-compare
|-- package.json
|-- package-lock.json
|-- playwright.config.js
|-- README.md
```

6. Test Coverage Summary

The project contains 120 functional test cases divided equally across 8 service modules.

Service Module	Test Cases
Authentication	15
Product and Search	15
Cart	15
Wishlist and Compare	15
User Profile and Account	15
Address and Shipping	15
Checkout and Payment	15
Customer Support and Information	15
Total	120

7. Cross-Browser Coverage

Each functional test case is executed across three Playwright browser projects.

Browser	Test Executions
Chromium	120
Firefox	120
WebKit	120
Total Cross-Browser Executions	360

8. Framework Design

The framework follows the Page Object Model pattern.

Key design points:

- Common reusable actions and assertions are stored in `pages/base.page.js`.
- Each service area has a dedicated page object class.
- Tests import shared fixtures from `fixtures/baseFixture.js`.
- Test files are grouped by service module under the `tests` directory.
- Test names include visible business-style test case IDs such as `AUTH-001`, `PROD-001`, and `CART-001`.

9. Page Object Classes

Page Object	Purpose
BasePage	Common navigation and assertion helper methods
AuthenticationPage	Login, register, password recovery, protected account redirects
ProductPage	Home page, categories, product details, search
CartPage	Cart page, cart controls, cart journey labels
WishlistComparePage	Wishlist, compare products, email a friend
UserProfilePage	Account, orders, addresses, profile-related links
AddressShippingPage	Shipping returns, availability, delivery, checkout progress
CheckoutPaymentPage	Checkout journey, gift card payment fields, price checks
CustomerSupportPage	Contact us, sitemap, policies, social links, newsletter

10. Fixtures and Optimization

The framework uses a custom Playwright fixture file:

```
fixtures/baseFixture.js
```

This fixture creates reusable page object instances for all service modules. It also uses a worker-scoped page to reduce repeated browser setup overhead and improve execution speed.

Optimization choices:

- Parallel execution enabled.
- Worker count configured for stable cross-browser execution.
- Video recording disabled.
- Trace recording disabled for normal local runs.
- Screenshots retained only on failure.
- Allure reporter enabled only when explicitly requested.

These changes reduce local execution time while keeping all required cross-browser coverage.

11. Playwright Configuration

The main configuration file is:

```
playwright.config.js
```

Important settings:

- `testDir: ./tests`
- `fullyParallel: true`
- `workers: 6`
- `timeout: 45000`
- `expect.timeout: 10000`

- baseUrl: <https://demowebshop.tricentis.com>
- Browsers: Chromium, Firefox, WebKit

12. NPM Commands

Command	Purpose
npm install	Install project dependencies
npx playwright install	Install Playwright browsers
npm run test:list	List all discovered tests
npm test	Run all tests across all browsers
npm run test:clean	Clean old reports and run all tests
npm run test:chromium	Run only Chromium tests
npm run report	Open Playwright HTML report
npm run test:allure	Run tests with Allure enabled and generate Allure report
npm run allure:open	Open generated Allure report

Recommended final execution command:

```
npm run test:clean
```

13. Reporting

The project supports two report types.

Playwright HTML Report

Generated automatically after a Playwright test run.

Open command:

```
npm run report
```

Allure Report

Run and generate Allure report:

```
npm run test:allure
```

Open Allure report:

```
npm run allure:open
```

14. GitHub Actions and GitHub Pages

The project contains a GitHub Actions workflow:

```
.github/workflows/allure-report.yml
```

Workflow purpose:

- Checkout repository
- Install Node.js dependencies
- Install Playwright browsers
- Run Playwright tests with Allure enabled
- Generate Allure report
- Publish the Allure report to the gh-pages branch

After successful workflow execution, GitHub Pages can host the Allure report from the gh-pages branch.

Expected Pages URL format:

```
https://<github-username>.github.io/<repository-name>/
```

15. Execution Result

The suite is designed to discover:

```
360 tests in 120 files
```

Expected full-run result:

```
360 passed
```

The user terminal execution showed all 360 tests passing. Any message such as "errors were not a part of any test" is related to Playwright browser cleanup after execution, not failed functional test cases. The recommended stable command is:

```
npm run test:clean
```

If WebKit cleanup issues appear on Windows, reduce workers:

```
npx playwright test --workers=4
```

16. Key Deliverables

Deliverable	Status
Playwright project setup	Completed
Page Object Model implementation	Completed
8 service modules	Completed
120 functional test cases	Completed
360 cross-browser executions	Completed
Playwright HTML report support	Completed
Allure report support	Completed
GitHub Actions workflow	Completed
GitHub Pages publishing support	Completed
Final project report	Completed

17. Conclusion

The Akshit Demo Web Shop Playwright Capstone project is complete. It provides automated functional coverage for an e-commerce demo website using Playwright, JavaScript, Page Object Model design, reusable fixtures, cross-browser execution, and report generation.

The project is ready for local execution, GitHub repository upload, CI execution, and Allure report hosting through GitHub Pages.